



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT 6

ROCK, PAPER, ROBOT

WHAT THIS PROJECT BUILDS

Show the camera a closed fist, a flat hand, or two fingers. The program reads the gesture, picks a move of its own, moves the arm to show it, and keeps score. Three systems have to agree for that to happen:

- **A hand tracker** that turns a picture of your hand into a number.
 - **A set of rules** that decides who won the round.
 - **An arm** that physically performs the robot gesture.
- *This is the first project where the arm is not decoration. If it does not move, the project is not finished.*

Project 5 was careful to keep the robot optional. This one is the opposite on purpose — but "the arm is essential" and "the program dies without the arm" are two different claims, and this project is about the gap between them.

THREE SYSTEMS, ONE LOOP

The main loop is short enough to hold in your head, and it does the same five things forever:

- 1 Get a frame**
From a background thread, not straight off the camera.
- 2 Find the hand**
MediaPipe returns landmarks, not an image of a hand.
- 3 Count and decide**
Fingers to a gesture, both gestures to a winner.
- 4 Move the arm**
Four and a half seconds of sleeping, in the middle of the loop.
- 5 Draw and show**
The picture you actually look at.

Step 4 is the one that surprises people. A blocking call in the middle of a real-time loop does not make the loop slower — it stops it completely. Everything in this project that feels wrong at first is downstream of those four and a half seconds.

A ROBOT THAT CAN BE ABSENT

The arm is essential to this project, but it is not essential to the program running. Those are different requirements, and the file is built to satisfy both.

```
# The robot handle. None means "no arm", and every use of it below is
# guarded, so the gesture game still runs on a machine with no robot.
mc = None

# Home pose. Used at startup and again at the end of every round, so it
# lives at module level rather than inside the try block below.
home_pos = [0, 0, 0, 0, 0, 0]
```

Two decisions in six lines. The handle starts as `None` so it always exists, whatever the hardware does. And the home pose moves out of the `try` block, because two functions need it and only one of them runs inside that block.

Anything created inside a `try` block may not exist when you reach the handler. Hoisting a shared constant out is the quiet fix that prevents a whole family of late, confusing crashes.

WHAT MEDIAPIPE HANDS YOU

Project 5 asked OpenCV for a colour. This project asks MediaPipe for a hand, and it gets back a list of 21 numbered points — one per finger joint, plus a wrist. The points come in a fixed order, every time, for every hand:

```
tips = [4, 8, 12, 16, 20] # Thumb, Index, Middle, Ring, Pinky tips
pips = [3, 6, 10, 14, 18] # Corresponding PIP joints
```

Two parallel lists. The thumb is 4, the index finger is 8, the middle is 12, and so on. Because the order never changes, you can count fingers by comparing numbers in a list instead of reasoning about a picture.

- *This is the whole reason to use a tracker. A shape recogniser gives you a label; a hand tracker gives you coordinates you can ask your own questions of.*

FRACTIONS, NOT PIXELS

Every landmark has `.x` and `.y` values, and they are not pixel coordinates. They are normalised: 0.0 is the left or top edge of the frame, 1.0 is the right or bottom edge. That single decision removes a whole class of bugs. The code below never converts to pixels, and it does not need to:

```
if hand_landmarks.landmark[tips[0]].x < hand_landmarks.landmark[pips[0]].x:  
    extended += 1
```

Because the values scale with the frame, the same comparison works at 640x480 and at 1280x720, and it still works if the hand moves toward the far side of the picture. Only the ordering is being tested — is this number smaller than that one — and ordering does not care about units.

WHY THE THUMB IS DIFFERENT

The thumb sticks out sideways. Comparing its tip to its joint on the `y` axis tells you almost nothing — a raised thumb and a thumb folded across the palm can sit at the same height. So the thumb gets its own test, on `x`:

```
# Thumb: compare x-coordinates (horizontal)
if hand_landmarks.landmark[tips[0]].x < hand_landmarks.landmark[pips[0]].x:
    extended += 1
```

The other four fingers all point the same way, so one loop covers them:

```
# Other 4 fingers: compare y-coordinates (vertical, tip above pip = extended)
for i in range(1, 5):
    if hand_landmarks.landmark[tips[i]].y < hand_landmarks.landmark[pips[i]].y:
        extended += 1
```

- *Note the loop starts at 1, not 0. The thumb was already counted by hand, so index 0 is skipped. An off-by-one here is the classic bug in this function, and it shows up as a hand that always reports one finger too many.*

COUNT, THEN GESTURE

Counting is done. Turning a count into a name is four branches and no cleverness at all:

```
if finger_count == 0:  
    return "Rock"  
elif finger_count == 5:  
    return "Paper"  
elif finger_count == 2:  
    return "Scissors"  
else:  
    return "Unknown"
```

Three numbers are meaningful and everything else is refused. That last branch is not laziness — it is what lets the main loop wait for a proper gesture instead of

- *Returning the string "Unknown" rather than a number or None is a good habit. Later code can test it against the list of real gestures with one `in` check and never has to guess what an unusual value meant.*

THE LIMIT OF COUNTING

Two fingers means Scissors. But which two?

```
# 2 fingers (index + middle) = Scissors
```

The comment claims index and middle. The code cannot tell. A peace sign and a finger-gun both report a count of two, and both are classified as Scissors.

- **Counting throws away identity** — it answers how many, never which.
- **The landmarks still know** — the tips list has a name for every finger, and nothing stops you testing each one individually.

Knowing what a limitation costs you is the difference between a program that works and one you understand. If identity mattered — say the robot had to point at a specific finger — you would compare `landmark[8]` against `landmark[12]` directly instead of counting.

WHY A PLAIN CAMERA FEELS LAGGY

Every project so far opened the camera and read it once per loop. That works fine. This project is different, because it adds **blocking** work to the loop — and that turns a small problem into an obvious one.

- *A camera does not deliver the frame you just asked for. It delivers the newest frame it has, and it keeps capturing while your code is busy. Those two facts are the whole problem.*

When your loop is fast, you read roughly the newest frame and nothing builds up. When your loop stalls for four seconds doing arm movements, the camera fills a buffer with four seconds of pictures. The next read hands you the oldest of them, not the newest.

So the lag you see is not the robot. It is the queue. The fix is not to make the loop faster — the loop has a genuine four and a half seconds of work in it — it is to stop the queue from ever building.

THE FIX: NEVER LET A QUEUE FORM

There are two ways to stop the queue building. One is to ask the driver for a smaller buffer. The class tries that first:

```
class ThreadedCamera:
    """Asynchronous camera reader thread to eliminate V4L2 frame buffer latency."""
    def __init__(self, src=0):
        self.cap = cv2.VideoCapture(src, cv2.CAP_V4L2)
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        # Reduce buffer size to minimize latency
        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
```

Look closely at that last line. It is passed a sensible value, it does not raise an error, and on many V4L2 setups it does nothing at all. Setting a property is not the same as the property taking effect.

- *Never trust a silent success. This one is worth remembering because it looks like it worked, and the lag it was supposed to remove is still there.*

The line is not wasted — it helps on backends that do honour it, and it documents the intent. But the real fix is the next slide.

A THREAD, A LOCK, AND A COPY

So the class pulls frames continuously on its own thread and hands over only the latest one. Three pieces make that safe:

```
self.lock = threading.Lock()
self.running = True
self.ret, self.frame = self.cap.read()

# Start background thread to continually pull frames from the V4L2 buffer
self.thread = threading.Thread(target=self._update, daemon=True)
self.thread.start()
```

The thread runs forever in the background. The main loop never waits for the camera; it just takes whatever the most recent frame happens to be. The queue still exists, but it is now exactly one frame deep.

- *The lock is not optional. Without it, the main loop could be halfway through drawing on a frame while the thread replaced it underneath.*

THE THREAD BODY

One method does all the reading, and it is short:

```
def _update(self):
    while self.running:
        if self.cap.isOpened():
            ret, frame = self.cap.read()
            if ret and frame is not None:
                with self.lock:
                    self.ret = ret
                    self.frame = frame
            time.sleep(0.005)  # Prevents high CPU usage on dedicated thread execution
```

Three details worth naming. The `self.running` flag is the only way this loop ever stops. The `with` block is what makes the assignment atomic. And the five-millisecond sleep stops the thread from spinning on a camera that cannot keep up — without it this is a busy loop that will heat the Pi.

WHY READ() HANDS BACK A COPY

Here is the method the main loop calls. The last line is the important one:

```
def read(self):  
    with self.lock:  
        if self.frame is None:  
            return False, None  
        return self.ret, self.frame.copy()
```

Your drawing code mutates the frame it is given.

`cv2.putText`, `cv2.rectangle` and `cv2.flip` all write straight into the image. If this returned the shared frame itself, every rectangle and every letter would be scribbled onto the frame the next detection pass is about to analyse.

- *The copy is what makes the shared frame safe. It costs a few milliseconds and it buys you a picture that is yours to draw on.*

SHUTTING THE THREAD DOWN

A background thread that is never stopped will keep the program alive. This is the shutdown, and it happens in the right order:

```
def release(self):  
    self.running = False  
    if self.thread.is_alive():  
        self.thread.join(timeout=0.5)  
    if self.cap.isOpened():  
        self.cap.release()
```

Flag first, then join, then release the camera. The join has a timeout, so even if the thread is stuck inside a blocking read the program still shuts down instead of hanging.

The daemon flag on the constructor is the safety net, not the plan. It means that even if `release()` is never called, Python will still exit. Relying on that alone is how you end up with a camera that stays locked after a crash.

WIN LOGIC AS A LOOKUP TABLE

Rock paper scissors is a bad fit for `if` chains. There are nine combinations and only six of them are wins, which is the wrong shape for branching. The program uses a dictionary instead:

```
WIN_MAP = {
    (0, 2): "PLAYER", (2, 0): "ROBOT",  # Rock > Scissors
    (1, 0): "PLAYER", (0, 1): "ROBOT",  # Paper > Rock
    (2, 1): "PLAYER", (1, 2): "ROBOT",  # Scissors > Paper
}
```

Each key is the pair of gesture indices, player first. Each value is who won. Two entries per line, six lines of behaviour total. Adding a fourth gesture would mean adding rows, not new branches.

The `.get()` call with a default of `"DRAW"` means an unknown pair can never raise a `KeyError` and crash the game mid-round. In this program the draw case is already handled earlier, so the default is genuinely unreachable — but it is the right defensive choice.

FOUR AND A HALF SECONDS OF SLEEP

Every round ends with the robot physically performing its gesture. Add up the sleeps in that function:

```
mc.send_coords(pose, ARM_SPEED, 1)
time.sleep(1.5)
# Set gripper
mc.set_gripper_state(grip, 80)
time.sleep(0.5)
...
time.sleep(1.0)
...
mc.send_angles(home_pos, ARM_SPEED)
time.sleep(1.5)
```

Four and a half seconds, all of it inside the main loop. Nothing is drawn during that time, so the window shows the same frozen frame until the arm is home.

- *The threaded camera keeps working throughout, so when the loop wakes up it gets a genuinely fresh frame. The picture is current. It just arrives more than five seconds late.*

TWO CLOCKS, AND WHY IT MATTERS

This program reads the time in two different ways, and the difference between them is the difference between a working feature and a dead one.

- **Once per frame** — the loop captures a timestamp near the top of each iteration, and everything later in that iteration reuses it.
- **At the moment you need it** — a fresh `time.time()` read at the point of use.

The first is cheaper and slightly stale — fine for measuring how long you have held a gesture, since being a frame late changes nothing. The second is correct whenever something slow happened in between:

```
cooldown_ready = (time.time() - last_game_time) > GAME_COOLDOWN
```

- *Had that line used the frame timestamp, the elapsed time would have measured from before the arm delay — and cooldowns are where stale clocks go wrong.*

DRAWING TEXT AT A FIXED POSITION

Almost everything on this screen is `cv2.putText`. The one technique worth noticing is that the position never moves while the content does:

```
if player_gesture != "Unknown":  
    cv2.putText(frame, f"Your Move: {player_gesture}", (20, 130),  
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)  
else:  
    cv2.putText(frame, f"Fingers: {finger_count} - Make a clear gesture!", (20, 130),  
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
```

Both branches draw at $y = 130$, so whichever one runs, the text lands in the same place. That is the trick for a line of text whose content changes: hold the position constant and swap the words. The two branches even use different sizes and colours, which is why the fallback line is readable at a glance while the confirmation line is easy

- *The fingers count only appears while the gesture is unrecognised, which makes it a genuinely useful debugging aid. It shows you the raw input exactly when the interpretation is failing.*

CLAIMS AND EVIDENCE

One habit runs through this whole project, and it is worth stating before you write any code.

- *Every message your program prints is a claim about how it behaves. The comment in your code is a claim about why. Both are untested until you check them.*

This project makes four claims about how it behaves — three of them printed on startup, one of them implied by the interface. None of them can be checked by watching the program start, because the program starts perfectly well whether they are true or not.

So the check has to be deliberate: read each claim, then write down what you would expect to see if it were true, then run it. If the two differ, you have found something. Most people never do this, because a program that starts up and does not crash feels like a program that

- *After the twenty steps we come back to those four claims and look at what this project actually did. All four were wrong.*

BEFORE YOU TYPE ANYTHING

Copy the starter file across and run it, exactly as it arrived:

```
scp M2-P6-Starter.py er@192.168.1.149:Documents
```

```
python M2-P6-Starter.py
```

This is the real output of the starter file with every `TODO` still empty:

```
Robot ready.  
Starting Rock-Paper-Scissors!  
Gestures: Fist=Rock, Open Hand=Paper, Peace Sign=Scissors  
Hold gesture steady for 1 second to play.  
Traceback (most recent call last):  
  File "M2-P6-Starter.py", line 208, in <module>  
    while cap.isOpened():  
NameError: name 'cap' is not defined
```

Every `TODO` marker you leave empty is a gap. The file still parses, starts, and prints its banner — then dies at the first line that needs something unwritten.

STEP 1 — IMPORT THE TOOLS

Six imports. Four of them you have used in every project so far:

```
import cv2
import mediapipe as mp
import numpy as np
import time
import random
import threading
```

Two of these are doing new work. `random` is what lets the robot pick a move, and `threading` is the whole answer to the lag problem. `numpy` comes along for the ride because MediaPipe hands back arrays.

Notice what is **not** imported. The `MyCobot280` class does not come from any of these — it comes from a library, and it gets its own line in the next step.

STEP 2 — IMPORT THE ARM

One import, on its own, because it is not part of the standard set of tools:

```
from pymycobot.mycobot280 import MyCobot280
```

Read that as: from the `pymycobot.mycobot280` module, take the `MyCobot280` class. The package lives on the Raspberry Pi, not in this file — which is why the import line has to be typed exactly as written.

A mistyped import fails immediately, at the top of the file, before a single line of your own logic has run.

- *That is a gift: the error message tells you the module is missing rather than leaving you to work out why the program stopped.*

It will also fail if you try to run this file on a laptop that has never seen the robot libraries installed. For this project the code has to be run on the Pi.

STEP 3 — CONNECT AND POWER ON

Five lines inside the `try` block that already exists. Two device paths, two short waits, and a confirmation line:

```
print("Connecting to myCobot280...")

mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)
mc.power_on()
time.sleep(0.5)
```

Read the constructor as two questions. Which serial port? `/dev/ttyAMA0`. At what baud rate? `1000000`. Both are wrong-looking numbers on purpose, and both must match the arm exactly.

The two `time.sleep(0.5)` calls are not padding — they are giving the board time to finish booting and to acknowledge the power command. Send commands faster than it can accept them and the very first one is the one that gets lost.

STEP 4 — GO TO THE HOME POSE

Two lines that tell the arm where to start and how long to settle there.

```
mc.send_angles(home_pos, 50)
time.sleep(2.0)
```

The two-second sleep is the longest single wait in the whole file, and it is at startup, where nothing is waiting on you yet. There is no reason to rush it.

- *Where is `home_pos` defined? You did not type it in this step. It is the line given to you near the top of the file, before the `try` block, because the main loop needs it too and the loop is nowhere near this code.*

The speed of 50 is deliberately slow. This is a starting position, not a demonstration — you want the arm to arrive without knocking anything over.

STEP 5 — SET UP MEDIAPIPE

This is the step that finally replaces colour detection with a hand tracker. Five lines of setup:

```
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)
```

- `static_image_mode=False` — a live video feed, not a photo, so keep the previous frame as a hint.
- `max_num_hands=1` — one pair of hands, which is all this game needs and all the loop can track.
- *Those last two confidence values are the only tuning knobs in the entire detection path. Raise them and the tracker gets faster but shakier; lower them and steadier but slower to notice a new hand.*

STEP 6 — OPEN THE CAMERA

One line, and it is not `cv2.VideoCapture`:

```
cap = ThreadedCamera(0)
```

The class is already written for you, above the `TODO` markers. The `0` is the camera index — zero is the first one. Everything that made the class complicated is a response to the four and a half seconds of blocking work that comes later in the loop.

- *Every call in the main loop stays the same as the other projects. `cap.read()`, `cap.isOpened()`, `cap.release()` — the class was written to be a drop-in replacement for a normal capture object, so the loop code never has to know there is a thread in here.*

STEP 7 — COUNT FINGERS, PART ONE

The function signature, the docstring, and the two lists of landmark indices that make this project possible:

```
def count_fingers(hand_landmarks):  
    """  
    Count extended fingers using MediaPipe landmarks.  
    Returns: number of extended fingers (0-5)  
    """  
    tips = [4, 8, 12, 16, 20] # Thumb, Index, Middle, Ring, Pinky tips  
    pips = [3, 6, 10, 14, 18] # Corresponding PIP joints  
  
    extended = 0
```

Index 0 of each list is the thumb, so the two lists stay parallel — position 2 is the middle finger in both. That is the property the rest of the function depends on.

STEP 7 — COUNT FINGERS, PART TWO

Two different comparisons, because there are two different kinds of finger:

```
# Thumb: compare x-coordinates (horizontal)
if hand_landmarks.landmark[tips[0]].x < hand_landmarks.landmark[pips[0]].x:
    extended += 1

# Other 4 fingers: compare y-coordinates (vertical, tip above pip = extended)
for i in range(1, 5):
    if hand_landmarks.landmark[tips[i]].y < hand_landmarks.landmark[pips[i]].y:
        extended += 1

return extended
```

- *The thumb is handled first and outside the loop, then the loop starts at 1. If it started at 0 the thumb would be counted a second time and every gesture in the game would be wrong by exactly one finger.*

Smaller `y` means higher up in the frame, so a tip above its joint means the finger is pointing up. That single inequality is the whole test.

STEP 8 — COUNT TO GESTURE

Four branches. The comments matter as much as the code, because they define what each pose is meant to be:

```
def classify_gesture(finger_count):  
    """  
    Classify gesture based on finger count.  
    Returns: "Rock", "Paper", "Scissors", or "Unknown"  
    """  
    # 0 fingers (fist) = Rock  
    # 5 fingers (open hand) = Paper  
    # 2 fingers (index + middle) = Scissors  
    # Anything else = Unknown  
    if finger_count == 0:  
        return "Rock"  
    elif finger_count == 5:  
        return "Paper"  
    elif finger_count == 2:  
        return "Scissors"  
    else:  
        return "Unknown"
```

- *Three counts are accepted and the fourth case is refused on purpose. Returning "Unknown" is how the main loop knows to keep waiting instead of scoring a round nobody made.*

STEP 9 — LET THE ROBOT CHOOSE

This is the entire function:

```
def get_robot_move():  
    """  
    Robot randomly chooses Rock, Paper, or Scissors.  
    Returns: gesture string  
    """  
    return random.choice(GESTURES)
```

It picks from `GESTURES`, the same list the main loop uses to test whether a gesture is valid, so the robot can never choose something the program does not understand.

- *No seeding. Every call draws a fresh move, so the robot is genuinely unpredictable — which is what makes the game worth replaying and what makes it worth fixing the cooldown.*

STEP 10 — DECIDE THE WINNER

Handle the tie first, then translate both names to indices and ask the lookup table:

```
def determine_winner(player_move, robot_move):  
    """  
    Determine round winner.  
    Returns: "PLAYER", "ROBOT", or "DRAW"  
    """  
    if player_move == robot_move:  
        return "DRAW"  
  
    p_idx = GESTURE_IDX[player_move]  
    r_idx = GESTURE_IDX[robot_move]  
  
    return WIN_MAP.get((p_idx, r_idx), "DRAW")
```

The tie is pulled out first because the dictionary has no entry for it. Leaving that in the table would mean filling all three tie pairs with "DRAW" for no benefit — and the early return also means the dictionary only ever holds the six interesting cases.

STEP 11 — THE GUARD COMES FIRST

The most important three lines in the project, and they are not about gestures at all:

```
def execute_robot_gesture(gesture):  
    global last_robot_move  
  
    # Record the move first, so the on-screen label is right even when  
    # there is no arm to move.  
    last_robot_move = gesture  
  
    # No arm attached: the gesture is still recorded and scored, it just  
    # is not performed. This is what lets the game run without hardware.  
    if mc is None:  
        return
```

- `global` — this function assigns to a variable the loop owns, so Python needs to be told.
- **Record first**, guard second. Reversing those two lines makes the on-screen label blank exactly when hardware is missing.

STEP 11 — THE MOVEMENT

Everything past the guard runs only when there is an arm. Choose the pose, move, hold, and come home:

```
if gesture == "Rock":
    pose = POSE_ROCK
    grip = GRIP_ROCK
elif gesture == "Paper":
    pose = POSE_PAPER
    grip = GRIP_PAPER
else: # Scissors
    pose = POSE_SCISSORS
    grip = GRIP_SCISSORS
# Move to gesture pose
mc.send_coords(pose, ARM_SPEED, 1)
time.sleep(1.5)
# Set gripper
mc.set_gripper_state(grip, 80)
time.sleep(0.5)
# Hold pose briefly for player to see
time.sleep(1.0)
# Return to home
mc.send_angles(home_pos, ARM_SPEED)
time.sleep(1.5)
```

- *The gripper is doing real work here: a closed gripper is a fist, an open one is a flat hand, and the two-finger pose is faked with the same open gripper. The arm is not pointing at a label, it is shaping a gesture.*

STEP 12 — DRAW THE UI, PART ONE

This is the longest function in the file — `draw_ui` — and it is only `cv2.putText` calls on one frame. The signature takes the data in and draws it, and reads nothing global except the scores:

```
def draw_ui(frame, player_gesture, robot_gesture, result, finger_count):
    """
    Draw game UI on frame.
    """
    h, w = frame.shape[:2]

    # Title
    cv2.putText(frame, "Rock Paper Scissors!", (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1.0, (255, 255, 255), 2)

    # Score
    score_text = f"Player: {player_score} | Robot: {robot_score} | Round: {rounds_played}"
    cv2.putText(frame, score_text, (20, 80),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)
```

Notice the asymmetry: the scores are read straight out of the enclosing scope without being passed in, while the result and the gestures arrive as parameters. That is inconsistent, and it is harmless, because assigning to a name like `game_result` inside this function would create a new local variable instead of updating the one the loop is about to read.

STEP 12 — DRAW THE UI, PART TWO

The player line, the robot line, and the one piece of information that makes this project debuggable:

```
# Player gesture detection
if player_gesture != "Unknown":
    cv2.putText(frame, f"Your Move: {player_gesture}", (20, 130),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
else:
    cv2.putText(frame, f"Fingers: {finger_count} - Make a clear gesture!", (20, 130),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

# Robot gesture
if robot_gesture:
    cv2.putText(frame, f"Robot Move: {robot_gesture}", (20, 170),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 0), 2)
```

- *That yellow fallback line is a debugging tool that ships in the final program. When the game refuses to score a round, this line tells you whether it read zero fingers or four.*

STEP 12 — DRAW THE UI, PART THREE

The verdict. Four outcomes, four colours, one line of output:

```
# Result
if result == "PLAYER":
    color = (0, 255, 0)
    text = "YOU WIN!"
elif result == "ROBOT":
    color = (0, 0, 255)
    text = "ROBOT WINS!"
elif result == "DRAW":
    color = (255, 255, 0)
    text = "DRAW!"
else:
    color = (255, 255, 255)
    text = game_result

cv2.putText(frame, text, (20, 230),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2, color, 3)
```

The fourth branch is the default prompt, and it reads `game_result` from the enclosing scope.

- *Whatever you do not recognise becomes the idle message.*

STEP 13 — READ A FRAME

At the top of the loop, five lines that are almost identical to previous projects:

```
ret, frame = cap.read()
if not ret:
    break

# Flip for mirror view
frame = cv2.flip(frame, 1)
h, w = frame.shape[:2]
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = hands.process(rgb_frame)
```

- `cv2.flip(frame, 1)` — horizontal flip, so moving right moves right on screen.
- `cvtColor` — MediaPipe expects RGB, OpenCV hands back BGR, and the tracker quietly performs badly if you skip this.

● *The `break` on a failed read is what stops this loop on a camera that has been unplugged mid-session, instead of spinning on `None` forever.*

STEP 14 — DRAW THE HAND SKELETON

Three lines of drawing, inside the hand loop, so the skeleton is overlaid on the picture you are about to analyse:

```
mp_drawing.draw_landmarks(  
    frame, hand_landmarks, mp_hands.HAND_CONNECTIONS,  
    mp_drawing.DrawingSpec(color=(0, 255, 0),  
        thickness=2, circle_radius=2),  
    mp_drawing.DrawingSpec(color=(255, 0, 0), thickness=2)  
)
```

Two `DrawingSpec` objects because MediaPipe draws the points and the bones separately, and they are styled independently. The connections themselves come from `HAND_CONNECTIONS` — the library already knows the bone layout, so you do not draw any of it by hand.

Draw before you count, not after. The landmark positions do not change because you drew on the frame, but if you ever start cropping the frame for display you must do it after this block, or the coordinates will no longer line up with the picture.

STEP 15 — COUNT AND CLASSIFY

Two lines, and they are the only two in the whole loop that talk to the functions you wrote:

```
finger_count = count_fingers(hand_landmarks)
player_gesture = classify_gesture(finger_count)
```

One counts, one names. Keeping them apart means you can test the second half of the project without a camera at all — call `classify_gesture(2)` directly and you get "Scissors", no hand required.

- *Note what is not here. Nothing checks that the count is stable, and nothing checks that this gesture is different from the last one. Two lines that look complete are not the same as two lines that are enough.*

This is the exact place where a real gesture game needs more than a counting function, and we will come back to it.

STEP 16 — ROBOT MOVE AND WINNER

Still inside the gesture check. Two calls, and the order matters:

```
robot_gesture = get_robot_move()  
result = determine_winner(player_gesture, robot_gesture)
```

The robot picks first, then the winner is computed from both moves. Swapping them computes the same answer, which is a good reminder that the winner function takes both moves rather than asking the robot.

Both lines sit inside the `if valid_gesture and cooldown_ready` condition, so neither runs unless the gesture has been held and the cooldown has expired. Those two flags are the entire rate limit for the game, and Step 15 never mentioned them.

STEP 17 — UPDATE THE SCORES

Straightforward, and worth reading closely because it decides whether the score and the round counter agree:

```
rounds_played += 1
if result == "PLAYER":
    player_score += 1
elif result == "ROBOT":
    robot_score += 1
```

- **rounds_played** counts rounds, not points — it increments on a draw too.
- **So the score line can read** 1 | 0 | Round: 3, and that is correct rather than a bug.

The reset key at the bottom of the loop resets all three together, which is why they belong together here.

STEP 18 — PERFORM THE GESTURE

One line. Everything expensive happens inside the function it calls.

```
execute_robot_gesture(robot_gesture)
```

- *This single line is where the loop stops for four and a half seconds. Nothing is drawn, nothing is detected, and nothing is checked. The gameplay logic is frozen for the entire duration of the arm movement.*

If the arm is not attached, Step 11 returned early and this line costs microseconds instead. So the four and a half seconds are not a property of the game — they are a property of the hardware.

Keep that in mind. It changes which of the problems ahead are always present and which only appear once you plug the arm in.

STEP 19 — DRAW THE GAME UI

Call the function you wrote in Step 12, near the bottom of the loop:

```
draw_ui(frame, player_gesture, last_robot_move, game_result, finger_count)
```

Five arguments, all of them values the loop already has. Note the second one: it passes `last_robot_move`, the label recorded in Step 11, not a live reading of the arm. The screen describes the last round, not the current pose.

- *This is also the last chance the verdict gets to appear. Whatever `game_result` holds at the moment this line runs is what the player sees — so the order of the lines below it is not a style preference.*

STEP 20 — SHOW THE FRAME

The last TODO in the file:

```
cv2.imshow("Mission 02 - Project 06: Rock Paper Scissors", frame)
```

Then the two lines that come with it. `waitKey(1)` is what makes the window interactive — without it the picture freezes and no key is ever read:

```
key = cv2.waitKey(1) & 0xFF
if key == ord('q'):
    break
elif key == ord('r'):
    player_score = 0
    robot_score = 0
    rounds_played = 0
    last_robot_move = None
    game_result = "Score reset!"
```

- *You have now filled in all twenty TODOs. The program is complete, it runs, and every message it prints is true. We are about to check that last part.*

READING THE SCREEN

The project is finished. Before changing a single line, learn to read what the program is already telling you.

- **The yellow fallback line** — how many fingers were counted. If it says 3, the problem is your hand, not your code.
- **The score line** — rounds played and points scored. If rounds climb but points do not, the robot is drawing every time.
- **The verdict line** — the biggest text on the screen, at 1.2 scale, and the one you will forget to look at.
- **The console** — three banner lines on startup, and the hardware warning block if the arm is not found.

● *Each of those is a measurement. A project that prints its own state is giving you the evidence you need before you have written a single print statement yourself.*

BEFORE YOU CHANGE ANYTHING

This project made four claims about its own behaviour. All four were wrong, and none of them was wrong in a way that crashes.

#	The claim	What actually happened
1	Hold steady for 1 second	A single noisy frame triggered a 4.5 second arm movement
2	The verdict appears	It was cleared before it was ever drawn
3	Rounds cannot repeat instantly	The cooldown was measured across the arm delay, so it never blocked
4	Runs without a robot	The handler crashed on a name it never defined

Three of those four are the same mistake wearing different clothes: a timestamp taken at the wrong moment.

HOW THIS PROJECT WAS CORRECTED

Every fix is already in the file you have. It starts with new state, because two of the four failures needed somewhere to remember when something happened:

```
GESTURE_HOLD_TIME = 1.0 # seconds
hold_gesture = None
hold_start = 0.0

# How long a verdict stays on screen before it is cleared
RESULT_DISPLAY_TIME = 2.0 # seconds
last_result_time = 0.0
```

Constant for the promise, constant for the verdict window, three variables that hold the time.

- **Fresh clocks** — `time.time()` is read at the point of use, and the verdict gets its own `last_result_time` instead of borrowing the cooldown stamp.
 - **A verdict window** — the result is stamped **after** the arm returns home, so the display window starts when it should.
 - **A defined handle** — `mc = None` before the `try`, assigned to `None` in the handler, guarded at both use sites.
- *Twenty lines changed in a 478 line file — no restructuring.*

CLAIM 1 — THE HOLD THAT WAS NOT

The program printed a promise:

```
print("Hold gesture steady for 1 second to play.")
```

And the loop that decides whether to play a round asked only whether the gesture was recognised. The version before this project was corrected gated on exactly one

```
valid_gesture = player_gesture in GESTURES
```

One frame could commit the arm, so the correction adds the question the promise implied:

```
if player_gesture in GESTURES:
    if player_gesture != hold_gesture:
        hold_gesture = player_gesture
        hold_start = current_time
        held_for = current_time - hold_start
    else:
        hold_gesture = None
        held_for = 0.0
valid_gesture = (player_gesture in GESTURES and held_for > GESTURE_HOLD_TIME)
```

Changing gesture resets the clock, an unrecognised one zeroes it — the second must be deliberate.

CLAIM 2 — THE VERDICT NOBODY SAW

A round finished, a winner was computed, the score updated, the arm moved. And the player never saw the result.

The round ended by stamping two values. Note that the timestamp comes from the top of the frame:

```
game_result = result
last_game_time = current_time
```

Four and a half seconds of arm movement later, the loop reached this, above the call that draws the screen:

```
# Clear the result message after a delay
if (time.time() - last_game_time) > RESULT_DISPLAY_TIME:
    game_result = "Make a gesture!"
```

The delay had already elapsed before the verdict was ever drawn, so every round reset the message on its way to the screen. The correction gives the verdict its own clock and stamps it after the arm has finished moving — because the move is what consumed the window.

CLAIM 3 — THE BROKEN COOLDOWN

A cooldown exists so that one gesture cannot fire several rounds in a row. This one tested the right idea against the wrong clock:

```
cooldown_ready = (current_time - last_game_time) > GAME_COOLDOWN
```

Both timestamps are frame timestamps. `last_game_time` was recorded at the top of the frame that started the round, and `current_time` is the top of a frame taken after four and a half seconds of blocking movement. The measured gap included the entire arm movement, so it was always larger than two seconds.

- *The cooldown was satisfied before it could ever refuse anything. The feature was present, named, commented and completely inert.*

The correction reads the clock at the point of use instead:

```
cooldown_ready = (time.time() - last_game_time) > GAME_COOLDOWN
```

CLAIM 4 — AN UNKEPT PROMISE

Pull the arm, run it on a laptop without the robot libraries, and the handler prints a promise:

```
print("Continuing without the robot. Gestures are still detected and scored.\n")
```

It could not: `mc` only existed inside the `try`, so the handler fell through to `finally`:

```
finally:  
    cap.release()  
    cv2.destroyAllWindows()  
    mc.release_all_servos()
```

- *NameError, on the one path guaranteed to work.*

The fix: `mc = None` before the `try`, guarded at the use.

```
finally:  
    cap.release()  
    cv2.destroyAllWindows()  
    if mc is not None:  
        mc.release_all_servos()
```

THE HABIT THAT CATCHES ALL FOUR

Different bugs, one repeated mistake: trusting that a value means what its name suggests at the moment you read it.

- **When was this clock read?** — a frame timestamp is fine for measuring across a frame and wrong for measuring across four seconds of sleeping.
- **Does this name exist yet?** — anything created inside a `try` block can be missing by the time you need it.
- **Is the promise still true downstream?** — a printed message makes a claim about the whole run, not about the line above it.

None of these four failures threw an error that pointed at the real problem. They were found by writing down what the program promised, then running it and looking for the promise. That is the same habit that found the OpenCV contour change in Project 5, and it is the one worth keeping.

- *A quiet bug is not a small bug. It is a bug that costs you the time to notice it.*

THE BRIEF

Rock, Paper, Robot

Your arm is playing rock paper scissors against a hand you cannot see. MediaPipe reads 21 landmarks off the camera, five comparisons turn those landmarks into a number, three numbers become a gesture, and a lookup table decides a winner.

Then the robot has to perform. Not describe. Perform — travel to the pose, hold it, come home, four and a half seconds of real movement between two frames.

Which means the whole program has to survive being wrong. Not crash. Survive.

YOUR MISSION

Three parts, in order.

- 1. Break it on purpose.** Remove the hold check from Step 15, run the game, and count how many rounds a single pass of your hand produces.
- 2. Break it on purpose, part two.** Unplug the arm, run the program, and record the exact text you get back.
- 3. Explain each one.** For each failure, write the claim it broke, the line that broke it, and the correction now sitting in the file.

Do this with the corrected file in front of you. The point is not to re-discover the bugs — they are already fixed — it is to prove to yourself that you can see a claim, break it, and name the fix without being told which line to look at.

● *A bug you cannot reproduce on demand is a bug you cannot prove you fixed.*

YOUR DATA LOG

Claim	How you broke it	What you saw	The line that did it
1 sec hold			
Verdict shows			
Cooldown holds			
Runs no-arm			

Exact text, not a paraphrase. The phrase `name 'mc' is not defined` is worth more to your future self than the word "crashed", because it tells you the failure happened in cleanup rather than in the game.

DEFINITION OF DONE

- ☐ All twenty TODOs filled in and the file runs on the Pi.
- ☐ No `TODO` markers left anywhere in the file.
- ☐ Each of the four failures above reproduced on demand, with the exact message recorded.
- ☐ The hold survives a two-second pause in tracking without firing a round.
- ☐ The verdict is on screen long enough to read, and the score line agrees with it.
- ☐ One gesture cannot start two rounds in a row.
- ☐ Pulling the arm mid-game leaves the program running.

- *The last one is the real test: the project keeps working with the arm unplugged.*

GO FURTHER

- **Four gestures.** Add Spock, and add the three new `WIN_MAP` pairs. Then decide what a count of 4 now means.
- **Two hands.** Raise `max_num_hands` to 2 and decide what the game is when both are playing.
- **Move the blocking work off the loop.** A thread for the arm, or a queue with one pending gesture, so the window never freezes.
- **Real velocities.** `send_coords` takes a speed argument. Find out what the units are and make the gesture feel deliberate rather than slow.
- **Check the tracker.** The confidence thresholds are guesses. Instrument them and find out which one is actually rejecting your hand.

Pick one. Not all five.

DEBRIEF

This project had three systems and one of them was optional, which turned out to be the interesting part. The camera problem was solved by never letting a queue form. The arm problem was solved by a four and a half second sleep nobody could remove. The bug problem was solved by treating a printed message as a promise and checking it.

Every one of the four failures was quiet. No traceback, no exception, no error message — just a score that went up, a verdict that never appeared, and a message that promised something the program could not deliver. Write the claim down first. Run it second.

That is the whole lesson, and it applies to every project in this module.